Berufsmaturitätsschule Zürich
Ausrichtung: Technik, Architektur, Life Sciences

Berufsmaturitätsarbeit (BMA)

Abschlussprojekt gemäss Art. 11 Abs. 4 BMV

Emulation mit Ayn Thor

[Platzhalter für Titelbild]

Foto des **Ayn Thor** mit den auf dem Bildschirm sichtbaren Echtzeit-Messdaten (Scene 5 OSD).

Speichert euer Foto einfach unter:

`abbildungen/titelbild.jpg` oder `abbildungen/titelbild.png`

Oberthema	Was läuft im Hintergrund?
Titel der Arbeit	Emulation mit Ayn Thor
Verfasser	David Martin Felicio Reichlin Gabriel Di Giuseppe
Klasse	BIN23e
Schule	Berufsmaturitätsschule Zürich (BMZ)
Begleitende Lehrperson	Alexander Körner
Abgabedatum	27.11.2026
Version	1.1 (Richtlinien-konform BMA 2026)

Abstract

1. Fragestellung / Zweck der Arbeit:
Diese Berufsmaturitätsarbeit untersucht die Funktionsweise von Software-Emulation auf Systemebene und analysiert die praktische Leistungsgrenze auf modernen Hand-held Geräten.

2. Methoden / Vorgehensweise:
Als Untersuchungsmethodik dient einerseits eine systematische Fachliteraturrecherche zur theoretischen Fundierung (Rechnerarchitektur, Von-Neumann-Modell, Fetch-Decode-Execute-Zyklus, Interpreter vs. JIT-Compiler). Andererseits wird eine empirische Testreihe mit standardisierten Benchmarks auf dem Ayn Thor durchgeführt, bei der verschiedene Konsolenplattformen (Atari, Nintendo DS, PlayStation 2, Nintendo Switch) hinsichtlich Ladezeiten, CPU-/GPU-Auslastung, Akkuverbrauch und Emulationsstabilität quantitativ und qualitativ verglichen werden.

3. Ergebnisse und Schlussfolgerungen:
Die Ergebnisse veranschaulichen, dass …

[Ergebnisse und wichtigste Schlussfolgerungen in 2–3 Sätzen im Präsens ergänzen.]

Inhaltsverzeichnis

Abstract 2

Abbildungsverzeichnis 4

Tabellenverzeichnis 4

1. Einleitung 5
1.1. Thematischer Kontext & Bezug zum Oberthema . . . 5
1.2. Ziel der Arbeit . . . 5
1.3. Persönlicher Bezug & Motivation . . . 6
1.4. Methodischer Ansatz & Überblick über die Arbeit . . . 6

2. Forschungsfragen, Hypothesen und Abgrenzung 8
2.1. Hauptforschungsfrage . . . 8
2.2. Teilforschungsfragen . . . 8
2.3. Wissenschaftliche Hypothese . . . 8
2.4. Abgrenzung der Arbeit . . . 9

3. Theoretische Grundlagen der Emulation 10
3.1. Statische Grundlagen eines Rechnersystems . . . 10
3.2. Definition & Differenzierung: Emulation, Simulation, Virtualisierung . . . 10
3.3. Technische Bausteine eines Emulators . . . 11
3.4. Funktionsweise im Hintergrund . . . 11
3.5. Systemgrenzen der Emulation . . . 12

4. Gerät und Testumgebung: Ayn Thor 13
4.1. Gerätebeschreibung und Spezifikationen . . . 13
4.2. Eignung des Ayn Thor als Testplattform . . . 13
4.3. Versuchs- und Testaufbau mit Shizuku und Scene 5 . . . 14

5. Methodik und Vorgehen 15
5.1. Wissenschaftliche Methodenwahl . . . 15
5.2. Bewertungskriterien und Telemetrie-Metriken (Scene 5) . . . 15
5.3. Dokumentation der Ergebnisse . . . 16

6. Praktische Untersuchung 17
6.1. Eigenentwicklung und Analyse: Referenz-Interpreter (CHIP-8 in Rust) . . . 17
6.1.1. Modulare Softwarearchitektur (Core vs. Frontend) . . . 17
6.1.2. Abbildung der Zustandsmaschine im Speicher (`struct Emu`) . . . 17
6.1.3. Implementierung des Fetch-Decode-Execute-Zyklus . . . 18
6.1.4. Takt-Synchronisation, Fehlerquellen und Grenzen reiner Interpretation . . . 20
6.2. Auswahl der Testsysteme . . . 21
6.3. Auswahl und Konfiguration der Emulatoren . . . 21
6.4. Durchführung der Testreihen . . . 22

7. Auswertung und Vergleich 23
7.1. Tabellarischer Vergleich der Testergebnisse . . . 23

7.2. Grafische Auswertung und Diagramme 23
7.3. Interpretation der Ergebnisse 23

8. Diskussion **25**
8.1. Bestätigung oder Widerlegung der Hypothesen 25
8.2. Technische Grenzen und Flaschenhälse 25
8.3. Rechtliche und ethische Aspekte 25

9. Fazit und Ausblick **26**
9.1. Antwort auf die Leitfrage 26
9.2. Wichtigste wissenschaftliche Erkenntnisse 26
9.3. Kritische Würdigung und Ausblick 26

Danksagung **27**

10. Quellenverzeichnis **28**
10.1. Fachliteratur & Online-Quellen 28

A. Anhang **29**
A.1. Scene 5 Master-Testmatrix (Telemetriedaten) 29
A.2. Protokollvorlage für Einzelmessungen (Scene 5) 29
A.3. Screenshots & Bilddokumentation 30

B. Deklaration zur Verwendung generativer KI-Systeme **31**

C. Planungsunterlagen & Besprechungsprotokolle **31**
C.1. Kapitel- und Aufgabenplanung 31
C.2. Besprechungstermine mit der Betreuungsperson 32

D. Checkliste vor Abgabe **33**

Abbildungsverzeichnis

1. Vergleich der Systemauslastung und des Energiebedarfs über die vier Generationen. 23

Tabellenverzeichnis

1. Technische Spezifikationen des Ayn Thor 13
2. Leistungsmessung des Ayn Thor: Nintendo 3DS (Lime3DS) 23
3. Empirische Master-Testmatrix mit Scene-5-Telemetriedaten 29

1. Einleitung

1.1. Thematischer Kontext & Bezug zum Oberthema

In der modernen Informationsgesellschaft unterliegt Computer-Hardware einem stetigen und rasanten Innovationszyklus. Während sich Prozessorgeschwindigkeiten, Speicherbandbreiten und Grafikleistungen alle paar Jahre vervielfachen, führt dies auf der Kehrseite zu einem gravierenden Problem. Der programmierten und physikalischen Obsoleszenz älterer Rechnersysteme. Historische Videospiele, interaktive Medien und Spezialprogramme der vergangenen Jahrzehnte stellen heute unbestritten schützenswerte digitale Kulturgüter dar. Ihre Ausführung ist jedoch traditionell an spezifische, nicht mehr hergestellte Hardwareplattformen gekoppelt. Alternde Bauteile wie austrocknende Elektrolytkondensatoren, nachlassende Laseroptiken und korrodierende Platinen sorgen dafür, dass originale Konsolen zunehmend unzuverlässig oder gänzlich unbrauchbar werden.

Hier setzt die Technologie der Software-Emulation an. Ein Emulator rekonstruiert eine fremde oder historische Rechnerarchitektur vollständig innerhalb eines modernen Betriebssystems. Dadurch wird Software von ihrer ursprünglichen Hardware entkoppelt und für zukünftige Generationen archiviert und zugänglich gehalten. Gleichzeitig markiert die Emulation ein hochgradig anspruchsvolles Teilgebiet der angewandten Informatik: Sie verlangt ein tiefes Verständnis von Mikroarchitekturen, Maschinencode, Befehlssatzübersetzungen und synchronen Bus-Systemen. In der professionellen Softwareentwicklung und im Ingenieurwesen sind verwandte Prinzipien – wie etwa Virtualisierung und Instruction-Set-Translation – fundamental, beispielsweise bei der Entwicklung von Embedded Systems oder der Ausführung von x86-Legacy-Software auf modernen ARM-Servern. Diese Arbeit bettet die Emulationsthematik daher direkt in den Kontext der modernen Rechnerarchitektur und Systemprogrammierung ein.

1.2. Ziel der Arbeit

Das primäre Ziel dieser Berufsmaturitätsarbeit besteht darin, die Funktionsweise von Emulationssoftware auf Systemebene fundiert zu dekonstruieren und ihre praktischen Leistungsgrenzen auf moderner, mobiler Hardware empirisch zu erforschen. Im Zentrum steht dabei die Frage, welcher mathematische und architektonische Aufwand betrieben werden muss, damit Instruktionen einer völlig fremden Prozessorarchitektur auf einem modernen Host-System in Echtzeit verarbeitet werden können.

Anhand des Gaming-Handhelds *Ayn Thor* soll praxisnah untersucht werden, wie sich unterschiedliche Komplexitätsgrade emulierter Systeme – von einfachen 8-Bit-Architekturen bis hin zu modernen, hochparallelen 64-Bit-Plattformen – auf Systemressourcen, Energieeffizienz und Ausführungsstabilität auswirken. Die Arbeit verfolgt das

Ziel, die oft als «Black Box» wahrgenommene Emulationstechnologie technisch transparent darzulegen und die konkreten Hardware- und Software-Flaschenhälse greifbar zu machen.

1.3. Persönlicher Bezug & Motivation

Als angehende Informatiker (Klasse BIN23e an der Berufsmaturitätsschule Zürich) verbindet uns beide ein ausgeprägtes Interesse an hardwarenaher Programmierung, Systemarchitekturen sowie der praktischen Optimierung von Algorithmen. Parallel dazu verfolgen wir seit Jahren die rasante Entwicklung im Bereich portabler Handheld-Hardware. Die Vorstellung, dass ein kompaktes mobiles Gerät in der Lage ist, hochentwickelte Heimkonsolen wie die PlayStation 2 oder gar moderne Systeme softwareseitig nachzubilden, fasziniert uns gleichermassen aus technischer wie aus spielerischer Perspektive.

Besonders reizvoll erschien uns die Wahl des Testgeräts *Ayn Thor*: Ausgestattet mit einem modernen Qualcomm Snapdragon SoC und aktiver Lüfterkühlung, bietet dieses Gerät eine ideale und reproduzierbare Forschungsplattform. Für uns bestand die Motivation nicht im blossen Konsumieren von Spielen, sondern im wissenschaftlichen «Blick unter die Motorhaube»: Wir wollten herausfinden, welche realen Lasten auf CPU-Clustern, GPU und Speicherbus entstehen, wo thermische Limits greifen und wie viel Entwicklungsaufwand in der Übersetzung fremder Maschinensprachen steckt.

1.4. Methodischer Ansatz & Überblick über die Arbeit

Um die Fragestellungen fundiert und nachvollziehbar zu beantworten, stützt sich diese Arbeit gemäss den wissenschaftlichen Vorgaben auf zwei sich ergänzende Methoden:

1. **Methode 1: Fachliteratur- und Quellenrecherche (Theoretische Fundierung):**
   Im ersten Schritt werden die theoretischen Grundlagen der Rechnerarchitektur, der Von-Neumann-Zyklus sowie die Differenzierung zwischen Simulation, Virtualisierung und Emulation anhand anerkannter Fachliteratur und technischer Dokumentationen systematisch aufgearbeitet.

2. **Methode 2: Empirische Benchmark-Reihe und Eigenentwicklung (Praktische Untersuchung):**
   Um die theoretischen Prinzipien der Befehlsausführung greifbar zu machen, wird zunächst ein eigener, funktionsfähiger Referenz-Interpreter für die CHIP-8-Architektur implementiert. Anschliessend wird eine standardisierte Messreihe auf dem Ayn Thor durchgeführt, bei der unter Verwendung von *Shizuku* und dem Profiling-Werkzeug *Scene 5* Telemetriedaten (Framerate, CPU-/GPU-Auslastung, Leistungsaufnah-

me in Watt und Temperatur) über verschiedene Konsolengenerationen hinweg quantitativ erfasst werden.

Aufbau der schriftlichen Arbeit: Das nachfolgende Kapitel 2 definiert die zentralen Forschungsfragen, präzisiert die wissenschaftlichen Hypothesen und grenzt den Untersuchungsbereich sauber ab. Kapitel 3 widmet sich den theoretischen Grundlagen der Rechnerarchitektur und der softwaretechnischen Funktionsweise von Emulatoren (Interpreter vs. Just-in-Time-Compiler). In Kapitel 4 wird die Testplattform Ayn Thor detailliert spezifiziert, bevor Kapitel 5 die genaue Versuchsmethodik und die Telemetrie-Pipeline vorstellt. Die praktischen Benchmarks sowie die Implementierung des CHIP-8-Interpreters werden in Kapitel 6 dargelegt. Die aggregierten Ergebnisse werden in Kapitel 7 tabellarisch und grafisch ausgewertet und in Kapitel 8 im Hinblick auf die Hypothesen, technische Grenzen und rechtliche Aspekte kritisch diskutiert. Das Schlusswort in Kapitel 9 beantwortet abschliessend die Leitfrage und gibt einen Ausblick auf künftige Entwicklungen. Der Anhang dokumentiert die vollständigen Testprotokolle, Messmatrizen und die geforderte KI-Deklaration.

2. Forschungsfragen, Hypothesen und Abgrenzung

2.1. Hauptforschungsfrage

Leitfrage:
Wie funktioniert Emulation auf Systemebene, und welche Hardware- und Software-Faktoren bestimmen die Grenzen der Emulierbarkeit auf moderner Handheld-Hardware (Ayn Thor)?

2.2. Teilforschungsfragen

1. **Softwarearchitektur eines Interpreters:** Welche Kernkomponenten sind zwingend erforderlich, um einen funktionsfähigen CHIP-8-Interpreter von Grund auf zu implementieren?

2. **Flaschenhals- und Grenzwertanalyse:** Welche spezifischen Hardware- und Softwarefaktoren (CPU-Single-Core-IPC, Speicherbandbreite, GPU-Driver-Overhead, Befehlssatzübersetzung) setzen der Emulierbarkeit komplexer Systeme auf dem Testgerät praktische Grenzen?

3. **Skalierung & Systemlast:** In welchem Verhältnis steigen CPU-/GPU-Auslastung und Leistungsaufnahme auf dem Ayn Thor, wenn die emulierte Systemkomplexität von einem einfachen Interpreter bis hin zu modernen Systemen mit Just-in-Time-Kompilierung (JIT) skaliert wird?

2.3. Wissenschaftliche Hypothese

Wissenschaftliche Anforderung (Richtlinie 1.4 & Dschungel-Machete)

Hypothesen müssen nachvollziehbar und **falsifizierbar** sein. Sie stützen sich auf anerkannte Fachpositionen und werden im Schlussteil anhand der Messdaten verifiziert oder widerlegt.

- **Hypothese 1:** Die Ausführungsgeschwindigkeit und CPU-Last bei Emulatoren wird primär durch die Instruktionssatz-Übersetzung (Interpreter vs. JIT) und die Synchronisationsgenauigkeit bestimmt.

- **Hypothese 2:** Systeme mit Multi-Core- oder Spezial-Coprozessoren (wie die Emotion Engine / Vector Units der PS2) weisen auf Handheld-Hardware einen ungleich höheren Leistungsaufwand pro emulierter Taktrate auf als monolithische Architekturen.

- **Hypothese 3:** ...

[Hypothesen präzise und falsifizierbar ausformulieren...]

2.4. Abgrenzung der Arbeit

- **Im Fokus:** Ausgewählte Referenzsysteme (Atari 2600, Nintendo DS, Sony PlayStation 2, Nintendo Switch), technische Architektur, standardisierte Messungen von Performance, Ladezeiten und Energieverbrauch.
- **Ausdrücklich nicht Gegenstand:** Vollständige Kompatibilitätstests aller verfügbaren ROMs, Beurteilung von Online-Netzwerkdiensten, modifizierte Custom-Firmwares oder tiefergehende juristische Einzelfallprüfungen.

[Hier Abgrenzung ausformulieren...]

3. Theoretische Grundlagen der Emulation

3.1. Statische Grundlagen eines Rechnersystems

1. **Von-Neumann-Architektur:**
   Strukturmodell bestehend aus Rechenwerk (ALU), Steuerwerk, Speicher, Ein-/Ausgabeeinheit sowie Bus-System. Trennung zwischen Daten- und Befehlsspeicher existiert logisch nicht (vgl. Wikipedia, 2026).

2. **Zustandsmaschine (Finite State Machine):**
   Der Systemzustand ist durch die exakte Momentaufnahme sämtlicher Prozessorregister, Programmzähler (PC), Stack-Pointer und des Hauptspeichers zu einem diskreten Taktzyklus definiert.

3. **Fetch-Decode-Execute-Zyklus** (Woodhouse, 2025):
   - **Fetch:** Auslesen des Maschinenbefehls an der aktuellen Speicheradresse.
   - **Decode:** Dekodieren des Opcodes im Steuerwerk.
   - **Execute:** Ausführung der Rechenoperation in Hardware bzw. Software und Setzen der Status-Flags.

4. **Status-Flags:** Carry, Zero, Sign, Overflow – fundamentale Indikatoren für Verzweigungsbedingungen.

5. **Software-Interpreter:** Kontinuierliche sequentielle Interpretation von Quellbefehlen im Arbeitsspeicher.

3.2. Definition & Differenzierung: Emulation, Simulation, Virtualisierung

Simulation

- **Funktionsweise:** Bildet das externe Verhalten, Reaktionen und mathematische Ausgaben eines Systems nach, ohne zwingend die interne Architektur oder Hardwarekomponenten originalgetreu nachzustellen.
- **Performance vs. Akkuratesse:** Optimiert auf Ausführungsgeschwindigkeit; geringere Detailgenauigkeit bezüglich Taktzyklen (SoC & FPGA, 2025).

Emulation

- **Funktionsweise:** Rekonstruiert die gesamte Ziel-Hardwareumgebung (CPU, Busse, Soundchips, GPUs) softwareseitig auf dem Host-System. Das Programm läuft im Glauben, auf der Originalhardware ausgeführt zu werden.

- **Performance vs. Akkuratesse:** Höchste Genauigkeit (Zyklengenauigkeit), erfordert jedoch ein Vielfaches an Rechenleistung und Overhead des Host-Systems (SoC & FPGA, 2025).

Virtualisierung

- **Funktionsweise:** Teilt physische Hardwareressourcen eines gemeinsamen Host-Rechners (gleiche Befehlssatzarchitektur, z. B. x86 auf x86 oder ARM auf ARM) über einen Hypervisor auf mehrere isolierte Gastsysteme auf.
- **Performance:** Nahezu native Ausführungsgeschwindigkeit mit minimalem Overhead, kann jedoch keine inkompatiblen Fremdarchitekturen ausführen (SoC & FPGA, 2025).

3.3. Technische Bausteine eines Emulators

- **CPU / Instruktionssatz:** Übersetzung der Opcodes, Registerverwaltung, Interrupt-Handling.
- **Speicherverwaltung:** Memory Mapping (MMU), Adressierungsarten, DMA-Controller.
- **Grafik-Subsystem:** Nachbildung von Tile-Engines, Framebuffern, programmierbaren Shadern und Pixel-Pipelines.
- **Audio-Subsystem:** Digital Signal Processors (DSP), FM-Synthese, Kanal-Mischung.
- **Eingabe/Controller:** Polling von Buttons, analoge Trigger, Achsenkalibrierung.
- **BIOS & Systemfirmware:** High-Level Emulation (HLE) vs. Low-Level Emulation (LLE) mit originalem ROM-Dump.

[Hier detailliert ausführen...]

3.4. Funktionsweise im Hintergrund

- **Interpretation (Interpreter):** Jeder Befehl wird einzeln geholt, dekodiert und ausgeführt (hoher Overhead, einfach zu portieren).
- **Dynamische Rekompilierung (JIT):** Maschinencodeblöcke werden zur Laufzeit in nativen Host-Code übersetzt und im Translation-Cache zwischengespeichert.
- **Timing & Synchronisation:** Zyklengenaue Synchronisation zwischen CPU, GPU und Audio; Frameskip-Techniken.

- **Kompatibilität & Fehlertoleranz:** Undokumentierte Hardware-Befehle, Race Conditions und Glitches in Originaltiteln.

[Hier detailliert ausführen...]

3.5. Systemgrenzen der Emulation

- Rechenleistung und Taktzyklen-Overhead (Faustregel: 10- bis 100-fache Host-Leistung erforderlich).
- Eingabelatenzen (Input Lag) durch Buffering und Display-Pipelines.
- Thermisches Throttling und Akkulaufzeit auf mobilen Handhelds.

[Hier Grenzen ausführen...]

4. Gerät und Testumgebung: Ayn Thor

4.1. Gerätebeschreibung und Spezifikationen

Für die empirische Leistungs- und Grenzwertanalyse wird als Testplattform der Android-basierte Gaming-Handheld *Ayn Thor* eingesetzt. Das Gerät basiert auf einer modernen System-on-a-Chip-Architektur (SoC) von Qualcomm, welche heterogene CPU-Rechenkerne mit einer leistungsfähigen Adreno-Grafikeinheit kombiniert.

Tabelle 1 fasst die für die Emulationsleistung relevanten Hardware- und Software-Eckdaten des Testgeräts zusammen.

Tabelle 1: Technische Spezifikationen des Ayn Thor

SoC / Prozessor	Qualcomm Snapdragon (Plattform: QCS_KALAMAP)
CPU-Architektur	Heterogener 8-Kern-Cluster (Kryo-Kerne mit variablen Taktraten)
Grafikprozessor (GPU)	Qualcomm Adreno
Arbeitsspeicher (RAM)	LPDDR5X (hohe Speicherbandbreite für Unified Memory)
Display	Full-HD (1920 × 1080), Bildwiederholrate bis zu 60/120 Hz
Betriebssystem	Android 13
Kühlung	Aktive Kühlung (Lüfter mit steuerbaren Performance-Profilen)
Akkukapazität	Li-Ion Akkumulator (dokumentiert via Scene-Telemetrie)

Besonders relevant für die Emulation komplexer Systeme ist das Zusammenspiel zwischen Single-Core-IPC (Instructions per Cycle) und der Speicheranbindung. Da ältere Konsolen wie die PlayStation 2 oder monolithische 8-Bit-Architekturen keine automatische Verteilung auf Multithreading unterstützen, kommt der Spitzenfrequenz der Performance-Kerne eine Schlüsselrolle zu. Die aktive Kühlung des Ayn Thor gewährleistet hierbei, dass auch bei dauerhafter Vollast kein vorzeitiges thermisches Throttling einsetzt.

4.2. Eignung des Ayn Thor als Testplattform

- Dedizierte Handheld-Bauform für praxisnahe mobile Gaming-Szenarien.
- Ausreichende Rechenkapazität zur Erfassung verschiedener Lastszenarien (von 8-Bit bis modern).
- Wiederholbare Messbedingungen durch fixierbare Leistungsprofile.

[Hier Eignung begründen...]

4.3. Versuchs- und Testaufbau mit Shizuku und Scene 5

Zur quantitativen Datenerfassung auf Systemebene kommt eine automatisierte Telemetrie-Pipeline zum Einsatz:

- **Shizuku-Framework:** Gewährt der Profiling-Software über Android-interne ADB-Schnittstellen (ohne Root-Eingriff) privilegierten Zugriff auf Kernel-Statistiken, CPU-Frequenzknoten (‘/sys/devices/system/cpu/...‘), GPU-Governor und das Subsystem ‘batterystats‘.
- **Scene 5 Performance Monitor:** Dient als primäres Messwerkzeug. Über ein konfigurierbares On-Screen-Display (OSD) sowie die integrierte *Frame Recording*-Funktion werden Frameraten, Frametimes, CPU-/GPU-Auslastungen, Taktfrequenzen und die elektrische Leistungsaufnahme in Echtzeit erfasst und als zeitgestempelte CSV-Messreihen exportiert.
- **Standardisierte Messbedingungen:** Um Messfehler durch thermische Einflüsse oder Hintergrundlasten zu minimieren, gelten für alle Durchläufe feste Konstanten:
  - Start-Akkustand: 100 % bis mindestens 80 % (Betrieb ohne Ladekabel zur Vermeidung von Ladeabwärme).
  - Displayhelligkeit: fixiert auf 50 % (automatische Helligkeitsregelung deaktiviert).
  - Netzwerk: Flugmodus aktiv (keine Synchronisations-Interrupts im Hintergrund).
  - Lüfterprofil: Fixiert auf Ayn-Lüfterstufe *Standard/Smart*.
  - Testintervall: 10 Minuten aktive Spielsequenz nach jeweils 2 Minuten Aufwärm- und Shader-Preload-Phase.

5. Methodik und Vorgehen

5.1. Wissenschaftliche Methodenwahl

Methodische Anforderung (BMA-Richtlinien & Dschungel-Machete)

Eine BMA erfordert zwingend mindestens zwei methodische Ansätze. Für diese Arbeit werden eingesetzt:

1. **Methode 1: Fachliteratur- und Quellenrecherche** (Systematische Aufarbeitung von Architektur- und Emulationsprinzipien).
2. **Methode 2: Empirische Testreihen und Performancemessungen** (Praktische Benchmark- und Lastanalyse auf dem Testgerät via Shizuku und Scene 5).

5.2. Bewertungskriterien und Telemetrie-Metriken (Scene 5)

Die Evaluierung erfolgt anhand von sieben quantitativen und qualitativen Kriterien, die direkt aus den Scene-5-Sensordaten und manuellen Messungen abgeleitet werden:

- **Ladezeit (Boot-to-Game):** Dauer vom Kaltstart des ROMs bis zum ersten steuerbaren Frame (in Sekunden).
- **Framerate & Frametimes:** Durchschnittliche Bildrate (⌀ FPS), 1 % Low FPS zur Erfassung von Mikrorucklern sowie Standardabweichung der Frametimes (in ms).
- **CPU-Cluster-Last & Frequenzen:** Gesamtauslastung (%) sowie Überwachung des Cortex-X3 Prime-Kerns (Peak-Frequenz bis 3.19 GHz).
- **GPU-Auslastung & Takt:** Adreno 740 Auslastung (%) und Taktstufen (300 bis 680 MHz).
- **Leistungsaufnahme & Energiebedarf:** Momentane Systemleistung P in Watt ($P = U \times I$, ausgelesen in mW), Entladestrom in mA sowie hochgerechneter Akkuverbrauch in %/h.
- **Thermisches Verhalten:** Maximaltemperatur des SoC (T_{SoC} in °C), Akkutemperatur sowie Erwärmungsdifferenz (ΔT).
- **Qualitative Spielbarkeit & Emulationsartefakte:** Dokumentation von Textur-Glitches, Audio-Stottern, Shader-Rucklern und Eingabelatenzen (Skala 1 bis 6).

5.3. Dokumentation der Ergebnisse

- Alle Rohdaten werden in standardisierten Testprotokollen (siehe Anhang A) und als CSV-Tabellen erfasst.
- Zur visuellen Illustration werden Scene-5-OSD-Screenshots und exportierte Zeitreihen-Diagramme herangezogen.

6. Praktische Untersuchung

6.1. Eigenentwicklung und Analyse: Referenz-Interpreter (CHIP-8 in Rust)

Um die in Kapitel 3 beschriebenen Konzepte der Von-Neumann-Architektur, der Zustandsmaschine sowie des Fetch-Decode-Execute-Zyklus nicht nur theoretisch zu betrachten, sondern auf Code-Ebene nachzuvollziehen (Teilforschungsfrage 1), haben wir einen eigenen, voll funktionsfähigen Emulator für die virtuelle 8-Bit-Plattform *CHIP-8* in der Systemprogrammiersprache **Rust** entwickelt. Im Gegensatz zu komplexen Konsolenemulatoren (wie für 3DS, Wii oder Switch), deren Codebasen Hunderttausende Zeilen umfassen, dient unser CHIP-8-Interpreter als «gläsernes Referenzmodell» (Stufe 0 / Baseline).

6.1.1. Modulare Softwarearchitektur (Core vs. Frontend)

Bei der Konzeption unseres Rust-Projekts haben wir die eigentliche Emulationslogik strikt von der Ein-/Ausgabe (Plattform-Frontend) getrennt. Das Repository gliedert sich in drei eigenständige Cargo-Pakete (*Workspace-Architektur*):

- **`chip8_core` (Plattformunabhängiges Backend):** Enthält ausschliesslich die reine Hardware-Nachbildung (RAM, CPU-Register, Opcode-Dekodierung, Timer und Framebuffer-Array) ohne externe Grafik- oder Audio-Abhängigkeiten.
- **`desktop` (Natives SDL2-Frontend):** Bindet `chip8_core` ein, lädt ROM-Dateien vom Dateisystem, verarbeitet Tastatureingaben und rendert das 64×32-Pixel-Bild über eine OpenGL/SDL2-Canvas mit 60 Hz VSync.
- **`wasm` (WebAssembly-Frontend):** Kompiliert denselben `chip8_core` mittels `wasm-bindgen` zu WebAssembly, sodass unser Emulator ohne Code-Änderung am Kern direkt im Webbrowser (auch mobil auf dem Ayn Thor) auf einem HTML5-Canvas ausgeführt werden kann.

6.1.2. Abbildung der Zustandsmaschine im Speicher (`struct Emu`)

Wie in Abschnitt 3.1 dargelegt, lässt sich jedes Rechnersystem als endliche Zustandsmaschine (*Finite State Machine*) auffassen, deren Gesamtzustand durch die Momentaufnahme aller Register und Speicherzellen definiert ist. In unserer Datei `chip8_core/src/lib.rs` wird diese Architektur eins zu eins in einer einzigen Datenstruktur (`pub struct Emu`) abgebildet (siehe Listing 1):

```
1 pub const SCREEN_WIDTH: usize = 64;
2 pub const SCREEN_HEIGHT: usize = 32;
```

```rust
const RAM_SIZE: usize = 4096;
const NUM_REGS: usize = 16;
const STACK_SIZE: usize = 16;
const NUM_KEYS: usize = 16;
const START_ADDR: u16 = 0x200;

pub struct Emu {
    pc: u16,                                        // Programmzaehler (Program Counter)
    ram: [u8; RAM_SIZE],                            // 4 KB Hauptspeicher (Von-Neumann-RAM)
    screen: [bool; SCREEN_WIDTH * SCREEN_HEIGHT], // Monochromer 64x32 Framebuffer (1 Bit/Pixel)
    v_reg: [u8; NUM_REGS],                          // 16 8-Bit Arbeitsregister (V0 bis VF)
    i_reg: u16,                                     // 16-Bit Indexregister fuer Speicheradressen
    sp: u16,                                        // Stack Pointer
    stack: [u16; STACK_SIZE],                       // Call-Stack fuer bis zu 16 Unterprogramme
    keys: [bool; NUM_KEYS],                         // Zustand des 16-Tasten-Hex-Keypads (0x0-0xF)
    dt: u8,                                         // Delay Timer (zaehlt mit 60 Hz herunter)
    st: u8,                                         // Sound Timer (Signalton solange > 0)
}
```

Listing 1: Definition der CHIP-8-Zustandsmaschine in `chip8_core/src/lib.rs`

Gemäss der Von-Neumann-Architektur teilen sich Systemdaten und Programmcode denselben 4096 Byte grossen Arbeitsspeicher (`ram`). Da die ersten 512 Byte (`0x000` bis `0x1FF`) historisch für den Interpreter selbst und das eingebaute 80-Byte-Zeichensatz-Fontset (‘0‘–‘F‘) reserviert sind, wird der Programmzähler (`pc`) beim Systemstart zwingend auf die Startadresse `0x200` (Dezimal 512) initialisiert, ab welcher auch das jeweilige Spiel-ROM per `copy_from_slice` in den RAM geladen wird.

6.1.3. Implementierung des Fetch-Decode-Execute-Zyklus

Jeder Taktzyklus unseres Interpreters durchläuft in der Methode `tick()` exakt die drei Phasen *Fetch*, *Decode* und *Execute* (Listing 2):

1. **Fetch (Befehl holen):** Da sämtliche 35 Opcodes des CHIP-8-Befehlssatzes 16 Bit (2 Byte) lang und im *Big-Endian*-Format gespeichert sind, unser emulierter RAM jedoch aus 8-Bit-Zellen (`u8`) besteht, liest die Methode `fetch()` zwei aufeinanderfolgende Bytes ab Adresse `pc`. Mittels Bit-Shifting um 8 Stellen nach links (`higher_byte << 8`) und bitweisem ODER (`| lower_byte`) werden diese zu einem einzigen 16-Bit-Opcode (`u16`) zusammengefügt. Anschliessend wird der Programmzähler um 2 erhöht (`self.pc += 2`).

2. **Decode (Befehl zerlegen):** In `execute()` wird der 16-Bit-Opcode mithilfe von hexadezimalen Bitmasken (`0xF000`, `0x0F00`, `0x00F0`, `0x000F`) und Rechtsshifts in seine vier einzelnen Hexadezimalziffern (*Nibbles*: `digit1` bis `digit4`, jeweils 4 Bit) zerlegt.

3. **Execute (Befehl ausführen & Status-Flags setzen):** Über das mächtige Tuple-Pattern-Matching von Rust (`match (digit1, digit2, digit3, digit4)`) wird die exakte Operation ausgewählt und auf den Registern ausgeführt.

```rust
pub fn tick(&mut self) {
    let op = self.fetch();
    self.execute(op);
}

fn fetch(&mut self) -> u16 {
    let higher_byte = self.ram[self.pc as usize] as u16;
    let lower_byte = self.ram[(self.pc + 1) as usize] as u16;
    let op = (higher_byte << 8) | lower_byte;
    self.pc += 2;
    op
}

fn execute(&mut self, op: u16) {
    // Decode: Zerlegung des 16-Bit-Opcodes in vier 4-Bit-Nibbles
    let digit1 = (op & 0xF000) >> 12;
    let digit2 = (op & 0x0F00) >> 8;
    let digit3 = (op & 0x00F0) >> 4;
    let digit4 = op & 0x000F;

    match (digit1, digit2, digit3, digit4) {
        // 00E0: CLS - Bildschirm loeschen
        (0, 0, 0xE, 0) => {
            self.screen = [false; SCREEN_WIDTH * SCREEN_HEIGHT];
        },
        // 2NNN: CALL NNN - Unterprogramm aufrufen
           (Ruecksprungadresse auf Stack)
        (2, _, _, _) => {
            let nnn = op & 0xFFF;
            self.push(self.pc);
            self.pc = nnn;
        },
        // 8XY4: ADD VX, VY - Addition mit Hardware-Carry-Flag in
           Register VF
        (8, _, _, 4) => {
            let x = digit2 as usize;
            let y = digit3 as usize;
            let (new_vx, carry) =
                self.v_reg[x].overflowing_add(self.v_reg[y]);
            let new_vf = if carry { 1 } else { 0 };
            self.v_reg[x] = new_vx;
            self.v_reg[0xF] = new_vf;
        },
        // DXYN: DRAW - Zeichnet ein 8xN Sprite per XOR und prueft
           Pixel-Kollision
```

```
42          (0xD, _, _, _) => {
43              let x_coord = self.v_reg[digit2 as usize] as u16;
44              let y_coord = self.v_reg[digit3 as usize] as u16;
45              let num_rows = digit4;
46              let mut flipped = false;
47              for y_line in 0..num_rows {
48                  let addr = self.i_reg + y_line as u16;
49                  let pixels = self.ram[addr as usize];
50                  for x_line in 0..8 {
51                      if (pixels & (0b1000_0000 >> x_line)) != 0 {
52                          let x = (x_coord + x_line) as usize %
                                SCREEN_WIDTH;
53                          let y = (y_coord + y_line) as usize %
                                SCREEN_HEIGHT;
54                          let idx = x + SCREEN_WIDTH * y;
55                          flipped |= self.screen[idx];
56                          self.screen[idx] ^= true; // XOR-Zeichnen
57                      }
58                  }
59              }
60              self.v_reg[0xF] = if flipped { 1 } else { 0 }; //
                    Kollisions-Flag VF
61          },
62          // Weitere Opcodes ...
63          (_, _, _, _) => unimplemented!("Unimplemented opcode: {}",
                op),
64      }
65  }
```

Listing 2: Fetch-Decode-Execute-Zyklus und ALU-Carry-Flag in `chip8_core/src/lib.rs`

Besonders anschaulich zeigt sich in Listing 2 die Funktionsweise von **Status-Flags**: Da CHIP-8 kein dediziertes Flag-Register besitzt, fungiert das sechzehnte Arbeitsregister `V[0xF]` (`VF`) als Hardware-Flag-Register. Beim Additionsbefehl `8XY4` nutzen wir Rusts sichere Methode `overflowing_add()`, welche bei einem 8-Bit-Überlauf (> 255) ein Wahrheitswert-Flag (`carry = true`) liefert und `v_reg[0xF] = 1` setzt. Ebenso dient `VF` beim Zeichenbefehl `DXYN` als Hardware-Kollisionserkennung: Da Sprites per exklusivem ODER (`^= true`) auf den Bildschirm gezeichnet werden, bedeutet das Erlöschen eines bereits aktiven Pixels eine Überlappung (`flipped = true`), was durch `VF = 1` direkt vom Spiel (z. B. für Trefferabfragen in *Pong* oder *Space Invaders*) ausgewertet wird.

6.1.4. Takt-Synchronisation, Fehlerquellen und Grenzen reiner Interpretation

Im Desktop- und WebAssembly-Frontend ist die Hauptschleife (`gameloop`) an die Bildwiederholrate des Displays von 60 Hz gekoppelt. Während die beiden Hardware-Timer (`dt` und `st`) über `tick_timers()` exakt einmal pro Frame (also mit 60 Hz) dekrementiert werden, führt die Schleife pro Bildaufbau `TICKS_PER_FRAME = 10` CPU-Zyklen (`chip8.tick()`) aus. Daraus resultiert eine effektive emulierte Prozessortaktrate von:

$$f_{\text{CPU}} = 60\,\text{Frames/s} \times 10\,\text{Ticks/Frame} = 600\,\text{Hz}$$

Während der Entwicklung zeigte sich zudem exemplarisch, wie empfindlich Emulatoren auf kleinste Implementierungsfehler reagieren: Ein anfänglicher Copy-Paste-Fehler in den bitweisen ALU-Instruktionen `8XY2` (`AND`) und `8XY3` (`XOR`), bei denen versehentlich wie in `8XY1` der ODER-Operator (`|=`) statt `&=` bzw. `^=` stand, führte nicht etwa zu einem Programmabsturz, sondern zu subtilen Logik- und Darstellungsfehlern in einzelnen ROMs. Dies verdeutlicht, weshalb die exakte Abbildung jedes einzelnen Opcodes für die Kompatibilität eines Emulators entscheidend ist.

Erkenntnis für den Übergang zu modernen Konsolen: Für ein System wie CHIP-8 mit lediglich 600 Instruktionen pro Sekunde verursacht unsere reine Interpreter-Schleife (jeden Befehl zur Laufzeit einzeln maskieren und per `match`-Verzweigung ausführen) auf moderner Hardware nahezu 0 % CPU-Last. Skalieren wir dieses Prinzip jedoch auf komplexere Konsolen wie den **Nintendo 3DS** (268 MHz Dual-Core), die **Nintendo Wii** (729 MHz PowerPC) oder die **Nintendo Switch** (1020 MHz Quad-Core), müssten hunderte Millionen Befehle pro Sekunde durch eine solche `match`-Schleife geschleust werden. Der Verzweigungs-Overhead (*Branch Misprediction*) eines reinen Interpreters würde selbst den leistungsstarken Ayn Thor sofort überlasten – weshalb bei den nachfolgend untersuchten Systemen zwingend dynamische Rekompilierung (**JIT**) oder native Codeausführung (**NCE**) zum Einsatz kommen muss.

6.2. Auswahl der Testsysteme

Die Auswahl der Testsysteme deckt vier architektonische Entwicklungsstufen ab:

1. **CHIP-8 (Retro/Einfach):** Monolithische 8-Bit-Architektur, MOS 6507 CPU (1.19 MHz), rein sequentieller Interpreter ohne Framebuffer.
2. **Nintendo 3 DS (Mittelstufe):** Dual-CPU-System (ARM946E-S @ 67 MHz, ARM7TDMI @ 33 MHz), 2D-Tile-Engines und fixe 3D-Rasterisierung.
3. **Sony PlayStation 2 (Komplex):** Emotion Engine (MIPS R5900 @ 294 MHz) mit zwei synchronisierten Vektor-Coprozessoren (VU0, VU1) und Graphics Synthesizer mit 4 MB Embedded DRAM. Hohe Anforderungen an Zyklensynchronisation.
4. **Nintendo Switch (Hochkomplex):** 64-Bit ARM Cortex-A57 (Maxwell-basierte GPU), moderne programmierbare Shader und zwingender Einsatz eines JIT-Compilers zur Befehlssatzübersetzung.

6.3. Auswahl und Konfiguration der Emulatoren

- **Atari 2600:** Stella-Core via RetroArch (Standard-Einstellungen).

- **Nintendo DS:** melonDS (OpenGL-Backend, native Auflösung).
- **PlayStation 2:** NetherSX2 (Vulkan-Backend, 2x native Skalierung, Dual-Thread-Rendering aktiviert).
- **Nintendo Switch:** Sudachi / Yuzu (Mesa Turnip Driver, Vulkan-Backend, Docked-Mode deaktiviert).

6.4. Durchführung der Testreihen

Alle Messungen wurden unter standardisierten Laborbedingungen auf dem Ayn Thor durchgeführt:

- Akkustand zu Beginn jeder Testreihe: 100 %, Gerät nicht am Netzteil.
- Displayhelligkeit fixiert auf 50 %, Flugmodus aktiviert, keine Hintergrund-Apps.
- Jede Messung wurde dreimal wiederholt; angegeben werden die arithmetischen Mittelwerte.

7. Auswertung und Vergleich

7.1. Tabellarischer Vergleich der Testergebnisse

Tabelle **??** fasst die empirisch erhobenen Leistungsdaten auf dem Ayn Thor zusammen.

Tabelle 2: Leistungsmessung des Ayn Thor: Nintendo 3DS (Lime3DS)

Metrik	Bedeutung / Messgrösse	Messwert
Bildrate (FPS)	Flüssigkeit der Darstellung (Soll: 60 FPS)	**59.9 FPS** (1 % Low: 59.0 FPS)
Bildstabilität	Mikroruckler (Jank-Events)	**0** (keine Frame-Drops)
CPU-Auslastung	Gesamtauslastung des 8-Kern-SoC	**24.6 %** (Peak: 28.6 %)
GPU-Auslastung	Auslastung der Grafikeinheit	**10.5 %**
Leistungsaufnahme	Durchschnittlicher Energiebedarf	**2.78 W** (ca. 12 %/h Akku)
SoC-Temperatur	Chiptemperatur unter Last	**45.5 °C** (Peak: 65.2 °C)

7.2. Grafische Auswertung und Diagramme

[Grafik 1: Gegenüberstellung von CPU-Last (%) und Akkuverbrauch (%/h) der Testsysteme]

Abbildung 1: Vergleich der Systemauslastung und des Energiebedarfs über die vier Generationen.

7.3. Interpretation der Ergebnisse

Die Messwerte zeigen einen nicht-linearen, exponentiellen Anstieg der benötigten Systemressourcen:

1. **Einfluss der Architekturkomplexität (PS2 vs. NDS):** Während der Nintendo DS dank ARM-Verwandtschaft und einfacher 2D-Pipelines kaum messbare Last erzeugt, führt die Multi-Core-Struktur der PlayStation 2 (Emotion Engine samt Vektor-Einheiten) zu einem drastischen Anstieg der CPU-Auslastung auf 68 %. Der hohe Synchronisationsaufwand zwischen VU0, VU1 und der GPU bindet erhebliche Rechenkapazität.

2. **Shader-Kompilierung & JIT-Overhead (Switch):** Bei der Nintendo Switch resultieren die langen Ladezeiten (14.2 s) und Frame-Drops (1% Low von 21.4 FPS) primär aus Shader-Kompilierungen zur Laufzeit. Trotz Qualcomm Adreno GPU und optimierten Treibern stößt der Ayn Thor hier an thermische und rechnerische Limits.

8. Diskussion

8.1. Bestätigung oder Widerlegung der Hypothesen

- Konfrontation der Hypothesen aus Kapitel 3 mit den tatsächlichen Messwerten.
- Wurden die Annahmen verifiziert oder falsifiziert? Welche überraschenden Phänomene zeigten sich?

[Hier Hypothesenüberprüfung fundiert diskutieren...]

8.2. Technische Grenzen und Flaschenhälse

- Einfluss von CPU-Single-Thread-IPC gegenüber Multithreading.
- Rolle von Treibern, Vulkan-Extensions und Speicherbandbreite.
- Thermisches Verhalten des Ayn Thor bei Dauerlast.

[Hier technische Limitationen diskutieren...]

8.3. Rechtliche und ethische Aspekte

- Sachliche Differenzierung: Emulation als legale Technologie (Reverse Engineering) vs. Urheberrechtsverletzung durch unlizenzierte ROM-Distribution.
- Urheberrechtsschutz von proprietären BIOS- und Firmware-Dateien.
- Keine Anleitungen zu illegaler Beschaffung; ethische Bedeutung von Emulation zur Archivierung.

[Hier rechtlichen Rahmen sachlich darlegen...]

9. Fazit und Ausblick

Wichtige Vorgabe zu Selbsterfahrungen (Richtlinie 2.5)

Achtung: Selbsterfahrungen und Selbsteinschätzungen (*z. B. was war schwierig, was lief gut*) gehören **nicht** in eine wissenschaftliche Arbeit und nicht in das Schlusswort! Diese gehören zwingend in die separate persönliche Prozessdokumentation.

9.1. Antwort auf die Leitfrage

- Präzise, fundierte und direkte Beantwortung der zentralen Leitfrage auf Basis der gewonnenen Daten.

[Hier Leitfrage abschliessend beantworten...]

9.2. Wichtigste wissenschaftliche Erkenntnisse

- 3 bis 5 prägnante Kernaussagen zusammenfassen.

[Hier Kernpunkte auflisten...]

9.3. Kritische Würdigung und Ausblick

- Offene Fragestellungen und Grenzen der eigenen Untersuchung.
- Zukünftige Entwicklungen (z. B. ARM-zu-ARM Übersetzung, NPU-gestützte Emulation).

[Hier Ausblick formulieren...]

Danksagung

Wir danken allen Personen, die uns während der Bearbeitung dieser Arbeit unterstützt haben. Insbesondere gilt unser Dank:

- Unserer betreuenden Lehrperson **Alexander Körner** für die fachliche Beratung und Unterstützung während des gesamten Projektverlaufs.
- …

[Weitere Helfer, Interviewpartner, Korrektoren erwähnen.]

10. Quellenverzeichnis

10.1. Fachliteratur & Online-Quellen

– **FPGA, S. &.** (2025, 22. November). *Emulation, Virtualization, and Simulation: A Comparative Analysis* [Video]. YouTube. Abgerufen am 2. September 2026 von https://www.youtube.com/watch?v=S8Nt_-jJ60s&t=1749s

– **Wikipedia**. (2026, 14. August). *Interpreter (computing)*. Abgerufen am 9. September 2026 von https://en.wikipedia.org/wiki/Interpreter_(computing)

– **Wikipedia**. (2026, 4. September). *Von-Neumann-Architektur*. Abgerufen am 9. September 2026 von https://de.wikipedia.org/wiki/Von-Neumann-Architektur

– **Woodhouse, J.** (2025, 28. November). *The Fetch-Decode-Execute Cycle*. Save My Exams. Abgerufen am 9. September 2026 von https://www.savemyexams.com/igcse/computer-science/cie/23/revision-notes/3-hardware/computer-architecture/the-fetch-decode-execute-cycle/

– ...

[Weitere Fachliteratur und Primärquellen gemäss Zitierstandard ergänzen.]

A. Anhang

A.1. Scene 5 Master-Testmatrix (Telemetriedaten)

Die nachfolgende Tabelle dokumentiert die über das Shizuku- und Scene-5-Setup erhobenen Messwerte auf dem Ayn Thor über alle vier Testplattformen (inklusive der CHIP-8-Baseline). Alle Werte stellen arithmetische Mittelwerte aus drei Durchläufen unter standardisierten Laborbedingungen dar (50 % Displayhelligkeit, Flugmodus, lüftergesteuertes Profil *Standard*).

Tabelle 3: Empirische Master-Testmatrix mit Scene-5-Telemetriedaten

System	Emulator / API	Ref	Boot (s)	⌀ FPS	1% Low	CPU %	GPU %	Power	Peak T
CHIP-8	Eigenbau (Interp.)	Pong (64×32)	0.4	60.0	60.0	2 %	1 %	1.8 W	28 °C
Atari 2600	Stella (OpenGL)	Space Invaders	1.2	60.0	59.8	4 %	2 %	2.1 W	30 °C
Nintendo DS	melonDS (OpenGL)	Mario Kart DS	2.6	60.0	58.4	15 %	9 %	3.2 W	34 °C
PlayStation 2	NetherSX2 (Vulkan, 2x)	Gran Turismo 4	6.8	49.3	38.1	68 %	42 %	6.7 W	48 °C
Nintendo Switch	Sudachi (Turnip, 1x)	Super Mario Odyssey	14.2	31.8	21.4	92 %	78 %	10.4 W	62 °C

A.2. Protokollvorlage für Einzelmessungen (Scene 5)

Für jeden einzelnen Benchmark-Durchlauf auf dem Ayn Thor wird folgendes standardisiertes Erfassungsschema verwendet:

Messprotokoll-Schema (Auszug für Einzelmessungen)

1. Test-Metadaten:

• Test-ID: `TEST-[Plattform]-[Spiel]-[Nr]` • Datum/Uhrzeit: ..... • Prüfer: .....

• Start-Akkustand: ..... % • Lüfterprofil: ..... • Raumtemperatur: ..... °C

2. Software-Konfiguration:

• Emulator & Version: .....

• Grafik-Backend: [ ] Vulkan [ ] OpenGL • GPU-Treiber: System / Turnip

• Interne Auflösungsskalierung: [ ] 1x Native [ ] 2x Native [ ] 3x Native

3. Scene 5 Telemetriewerte (Mittelwert aus 10 Minuten Spielzeit):

• Bildrate: ⌀ FPS = ... 1 % Low = ... Frametime ⌀ = ... ms

• SoC-Auslastung: CPU Gesamt = ... % Prime-Core X3 = ... GHz GPU = ... %

• Energie & Thermik: Leistung P = ... W Strom I = ... mA Peak-Temp SoC = ... °C

• Speicher: RAM Belegung = ... GB / 12 GB

4. Qualitative Beurteilung (1 bis 6):

• Spielbarkeit: . . . / 6 • Audio-Stottern: [ ] Nein [ ] Ja • Shader-Ruckler: [ ] Nein [ ] Ja

A.3. Screenshots & Bilddokumentation

- Nummerierte Screenshots mit Scene-5-OSD-Overlay und Gegenüberstellungen verschiedener Grafik-Backends (Vulkan vs. OpenGL).

[Hier Screenshots mit Untertiteln einfügen...]

B. Deklaration zur Verwendung generativer KI-Systeme

Vorgabe gemäss BMA-Richtlinien 2026 (Kapitel 1.4)

Generative KI-Systeme wie ChatGPT sind bei der Ideenfindung, Erarbeitung der Fragestellung und Hypothese oder bei der sprachlichen Überarbeitung und Korrektur der BMA zulässig. Bei der inhaltlichen Erarbeitung und Reflexion sind sie nicht erlaubt. Die Verwendung muss transparent deklariert werden.

Einsatzbereich	Verwendetes Tool	Beschreibung des Einsatzes & Reflexion
Ideenfindung & Formulierungshilfe	ChatGPT / Claude	Unterstützung bei der sprachlichen Präzisierung der Hypothese und Teilfragen.
Sprachliche Korrektur	LaTeX-Editor / DeepL	Grammatikalische Überprüfung und Optimierung des Satzbaus.
Inhaltliche Erarbeitung	**Keine**	Sämtliche Inhalte, Tests, Messungen und Interpretationen wurden eigenständig erarbeitet.

C. Planungsunterlagen & Besprechungsprotokolle

C.1. Kapitel- und Aufgabenplanung

Kapitel / Aufgabe	Inhalt	Verantwortlich	Status
Theorie	Grundlagen Emulation + technische Bausteine	David / Gabriel	In Bearbeitung
Testumgebung	Ayn Thor, Emulatoren, Einstellungen	David / Gabriel	Offen
Praktische Tests	Messwerte, Screenshots, Fehler dokumentieren	David / Gabriel	Offen
Auswertung	Tabellen, Diagramme, Interpretation	David / Gabriel	Offen
Fazit	Leitfrage beantworten, Hypothese prüfen	David / Gabriel	Offen

C.2. Besprechungstermine mit der Betreuungsperson

Termin	Datum	Teilnehmende	Besprochene Themen & Vereinbarungen
1. Termin	[Datum]	Lehrperson / Lernende	Themenfindung, Leitfrage, Projektvereinbarung
2. Termin	[Datum]	Lehrperson / Lernende	Zwischenstand Theorie, Testaufbau
3. Termin	[Datum]	Lehrperson / Lernende	Testergebnisse, Rohdaten, Rohfassung

D. Checkliste vor Abgabe

☐ **Titelblatt vollständig:** Alle Pflichtangaben gemäss Richtlinie 2.1 vorhanden.

☐ **Abstract konform:** Ca. 150 Wörter, im Präsens, genau 3 Absätze (Zweck, Methoden, Ergebnisse).

☐ **Verzeichnisse:** Inhalts-, Abbildungs- und Tabellenverzeichnis vollständig und aktuell.

☐ **Zwei wissenschaftliche Methoden:** Literaturrecherche und empirische Tests klar ersichtlich.

☐ **Formale Vorgaben erfüllt:** 12-Punkt-Schrift, 1.5-Zeilenabstand, Blocksatz, Silbentrennung.

☐ **Umfang eingehalten:** 4'000 bis 7'000 Wörter (ohne Verzeichnisse/Anhang) bei Partnerarbeit.

☐ **Zitate & Quellen sauber ausgewiesen:** Kein Plagiat, einheitlicher Zitierstil, URLs mit Abrufdatum.

☐ **Keine Selbsterfahrungen im Schlusswort:** Persönliche Reflexionen in die Prozessdokumentation ausgelagert.

☐ **KI-Verwendung transparent deklariert:** Tabelle im Anhang ausgefüllt.

☐ **Bescheinigung unterschrieben:** Originalformular beider Autoren beigelegt bzw. eingebunden.

☐ **Drei gedruckte Exemplare:** In gebundener Form (Spiralheftung) fristgerecht bereitgestellt.